Exact Recursion 方法教学主稿

从最基础的随机游走公式，到主线 one-target 机制稿中的实际计算

2026 年 4 月 15 日

结论先行

这份文稿想做一件很具体的事：把本仓库主线中真正大量使用的 *Exact Recursion*（也常叫 time recursion、时域递推、精确吸收递推）从头讲清楚，而且假设读者只记得非常基础的线性代数和微积分。

先说最短答案。 在你的研究里，最重要的不是“随机挑很多 Monte Carlo 轨迹看图像”，也不是“默认先把所有问题都丢进 Giuggioli/Sarvaharman 一类的生成函数 family 里求解”。真正的主求解器，尤其是在当前二维主线里，是：先把问题写成一个有限状态的吸收 Markov 链，再把**尚未命中目标的概率质量**一步一步精确推进。每一步流入目标点的那部分概率，就是这一时刻的首达时间概率。

为什么这个方法重要。 它重要有三个原因：

1. 它直接输出完整的首达时间分布 $f(t)$，所以你能看到早峰、谷底和晚峰；
2. 它不是 Monte Carlo 采样，因此没有统计噪声；只要转移概率写对了，给定时间窗内的结果就是确定性的；
3. 它和你的机制分析天然兼容，因为同一套递推不仅能给总曲线，还能给通道分解、窗口条件化、timing observables 和 budget observables。

这份稿子和仓库主线的关系。 本文会一直拿 `grid2d_one_two_target_gating` 的 one-target shared symmetric baseline 当主例子。因为在当前仓库语言里，最核心的一句话已经很明确：标准配置是 shared symmetric baseline；`peak1` 是 corridor fast branch；`peak2` 的主体是 left-open detour/rollback branch；膜穿越只是慢支路的次级修饰，而不是主角。换句话说，Exact Recursion 在这里不只是“算了一条曲线”，而是把整套机制语言撑起来了。

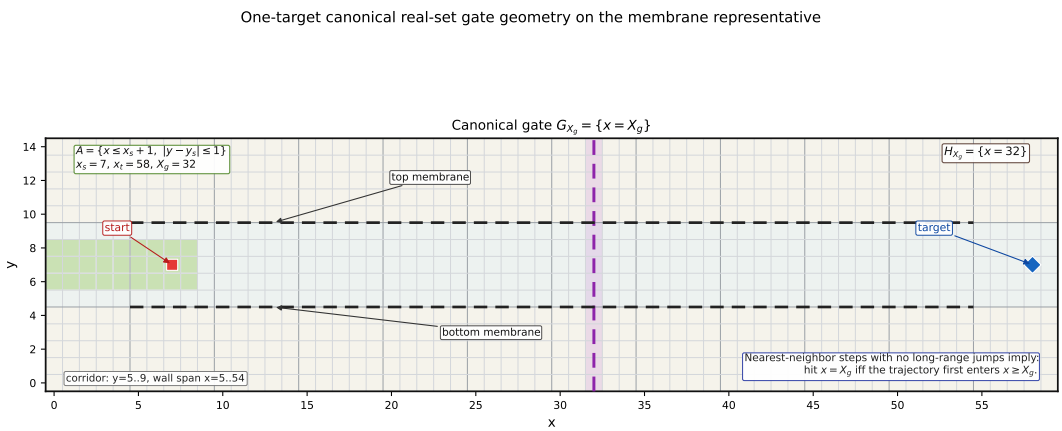

图 1：当前主线 one-target 例子的标准几何图。本文会一直围绕这张图来解释：状态空间是什么、目标是什么、哪些区域对应快支路、哪些区域对应慢支路。

1 最基础的起点：什么是离散时间随机游走

先把问题说得最朴素一点。 我们关心一个粒子（或者 walker）在离散的时间步 $t = 0, 1, 2, \ldots$ 上运动。每一步，它会以某些概率走到相邻格点，或者在 lazy 模型里原地停留。把“当前位置”记作 X_t，这就是一个最基本的离散时间随机过程。

什么是状态。 对二维格点模型来说，一个状态通常就是一个格点坐标，例如 (x, y)。如果格子大小是 $N \times N$，那全部状态就是所有合法的 (x, y)。如果后面我们再给轨迹附加标签，例如“是否已经 leak”“是否已经回到起始盆地”，那状态空间会更大，但思想完全一样：**状态就是系统此刻所处的全部信息**。

什么是转移概率。 如果当前在状态 i，下一步走到状态 j 的概率记作 P_{ij}。所有这些概率拼起来就是转移矩阵 P。这里最容易忘掉的一点是：矩阵里的每个元素不是一个数列，而是“一步转移”的概率。也就是说，矩阵负责回答“下一步怎么走”，而不是“最终会不会命中”。

什么是 Markov 性。 Markov 性的意思是：在知道当前状态以后，下一步怎么走只依赖当前状态，不依赖更早以前的历史。它并不意味着系统“没有记忆”，而是意味着只要你把需要的记忆都塞进状态定义里，**当前状态就已经包含了未来一步所需的全部信息**。

二维随机游走的最基本主方程。 先看最简单的、无偏、非 lazy 的二维随机游走。若

$$p_t(x, y) = \mathbb{P}(X_t = (x, y)),$$

那么在内部点上有

$$p_{t+1}(x, y) = \frac{1}{4}p_t(x-1, y) + \frac{1}{4}p_t(x+1, y) + \frac{1}{4}p_t(x, y-1) + \frac{1}{4}p_t(x, y+1). \tag{1}$$

这条式子读法很简单：要想在第 $t+1$ 步站到 (x, y)，那第 t 步必须先在四个邻居之一，而且从那个邻居走到这里。四条可能路径的概率加起来，就是新的位置分布。

为什么矩阵乘法会自然出现。 如果把所有格点按某个固定顺序排成一个长向量

$$p_t = \big(p_t(s_1), p_t(s_2), \ldots, p_t(s_M)\big),$$

那上面的每个“从哪些邻居流进来”的式子其实都可以统一写成

$$p_{t+1} = p_t P \quad \text{或} \quad p_{t+1} = P^\top p_t, \tag{2}$$

具体写成左乘还是右乘，只取决于你把 P_{ij} 定义成“从 i 到 j”还是“从 j 到 i”。仓库里两种写法都出现过，但物理内容是一样的：**矩阵乘法只是把很多个“流入求和”方程压缩成一条紧凑写法**。

lazy 和 bias 在这里怎么进来。 如果模型是 lazy 的，那每一步除了向邻点走，还允许以一定概率原地停留。最常见的写法是引入参数 $q \in (0,1]$ 表示“本步真的去移动的总概率”，那么原地停留概率就是 $1-q$。如果再引入方向偏置，例如向右更容易，那左右上下四个方向的权重就不再均匀，而是按照全局偏置、局部箭头、sticky site 和 barrier/membrane 等规则重新分配。仓库里的二维主线就是这种“lazy + bias + 局部结构”的一般形式。

2 从位置分布到首达时间

位置分布不是我们真正想要的量。 只知道“第 t 步时粒子在什么地方”的分布还不够。你的研究真正关心的是：**粒子第一次命中目标是在什么时候**。这就是首达时间（first-passage time）。

首达时间怎么定义。 设目标集合为 T_{set}。首达时间定义为

$$T = \inf\{t \geq 1 : X_t \in T_{\text{set}}\}. \tag{3}$$

如果目标只有一个格点 $(x_\star, y_\star)$，那就是“第一次走到这个目标格点的时刻”。如果目标是一条对角线、两个 target 的并集，或者 pair chain 里的 encounter set，定义形式完全一样，只是 T_{set} 更大。

三个最重要的分布量。 一旦定义了 T，我们就能写出：

$$f(t) = \mathbb{P}(T = t), \tag{4}$$

$$F(t) = \mathbb{P}(T \leq t), \tag{5}$$

$$S(t) = \mathbb{P}(T > t). \tag{6}$$

其中 $f(t)$ 是首达时间的 pmf，也就是你论文里真正画双峰的那条曲线；$F(t)$ 是累计命中概率；$S(t)$ 是到第 t 步为止还没命中的概率，也叫 survival。

为什么“在目标点上的概率”不等于“首次命中的概率”。 这是整篇文稿最关键的概念之一，值得重复说。

若你只是看普通位置分布 $p_t(\text{target})$，那它统计的是“第 t 步在目标上”的概率。这会把两种轨迹混在一起：

- 恰好在第 t 步第一次命中目标的轨迹；
- 更早就已经到过目标、后来还继续算在目标上的轨迹。

而首达概率 $f(t) = \mathbb{P}(T = t)$ 只允许第一类轨迹，绝不允许第二类轨迹重复计数。所以 first-passage 的核心工作，不是普通传播，而是要把已经命中过的概率质量**从后续传播里剔除出去**。

吸收态和瞬态态。 最自然的做法是把目标当成吸收态。所谓吸收态，就是一旦到达就不再离开。与之相对，其余还会继续传播的状态叫瞬态态（transient states）。从 first-passage 的角度看，我们真正要推进的是瞬态态上的概率质量，因为一旦某条路径第一次进入吸收态，它的首达时间已经定下来了，不应该再参与后续的“未命中传播”。

3 Exact Recursion 的核心想法

这套方法到底推进什么。 Exact Recursion 推进的不是普通位置分布，而是

$$u_t(s) = \mathbb{P}(X_t = s,\ T > t), \tag{7}$$

也就是：“到第 t 步为止还没有命中目标，并且此刻位于状态 s 的概率。”

为什么这个量更合适。 因为一旦我们知道了所有状态上的未吸收质量 $u_t(s)$，就能同时拿到三件东西：

1. 下一步还没命中的分布 u_{t+1}；
2. 下一步首次命中的概率 $f(t+1)$；
3. 当前 survival $S(t) = \sum_s u_t(s)$。

也就是说，**同一份递推同时控制了曲线和生存函数**。

把状态空间拆成两部分。 把状态排成“瞬态态在前，吸收态在后”的顺序后，转移矩阵可以分块写成

$$P = \begin{pmatrix} Q & R \\ 0 & I \end{pmatrix}. \tag{8}$$

这里：

- Q 是瞬态态到瞬态态的一步转移；
- R 是瞬态态到吸收态的一步流量；
- 右下角的 I 表示一旦进入吸收态就留在那里。

对单目标问题，R 其实可以压成一列向量 r，因为“吸收态”只有一个目标点。

最核心的递推公式。 若当前未吸收分布写成行向量 u_t，那么 Exact Recursion 的母式就是

$$u_{t+1} = u_t Q, \tag{9}$$

$$f(t+1) = u_t r, \tag{10}$$

$$S(t) = u_t \mathbf{1}. \tag{11}$$

这三条式子就是整套方法的真正核心。后面你在二维主线里看到的一切：总曲线、双峰窗口、通道质量、window-conditioned observables，都是建立在它之上的。

每条式子在说什么。

- 式 (9) 说：下一步仍然未命中的概率质量，只来自当前未命中的质量通过瞬态到瞬态的边继续流动。
- 式 (10) 说：下一步首次命中的质量，就是当前未命中质量通过瞬态到目标的边流出去的总量。
- 式 (11) 说：survival 就是当前所有未命中质量加总。

为什么这三条式子是严格正确的。 原因其实只用到两件事：全概率公式和 Markov 性。

对任一瞬态态 j，有

$$u_{t+1}(j) = \mathbb{P}(X_{t+1} = j,\, T > t+1).$$

要想在第 $t+1$ 步还没命中而且站在 j，那么第 t 步一定已经处于某个瞬态态 i，并且当时还没命中。由 Markov 性，下一步从 i 走到 j 的概率只取决于 i 本身，所以

$$u_{t+1}(j) = \sum_i \mathbb{P}(X_t = i,\, T > t)\, Q_{ij} = \sum_i u_t(i) Q_{ij}.$$

这就是 (9)。

同理，若 r_i 表示“从瞬态态 i 下一步直接命中目标”的概率，那么

$$f(t+1) = \sum_i u_t(i) r_i,$$

这就是 (10)。所以它不是近似方法，而是在**精确统计所有长度为 t 的未命中轨迹，在下一步流向哪里**。

把它写成矩阵幂。 从 (9) 出发反复递推可得

$$u_t = u_0 Q^t. \tag{12}$$

因此

$$f(t+1) = u_0 Q^t r. \tag{13}$$

如果你更习惯列向量写法，也可以写成

$$p_{t+1} = Q p_t, \qquad f(t) = r^\top p_t,$$

这就是仓库里多个文稿使用的等价口径。

为什么“每步把 target 概率拿掉”正是 first-passage 的关键

这句话很值得单独成节，因为你最近反复问到它。

先想一个错误做法。 如果我们每一步都按普通位置分布推进，但不把目标点概率拿掉，那么一条在第 5 步第一次命中目标的轨迹，在第 $6, 7, 8, \ldots$ 步还会继续留在目标点上。这样一来，后续每一步的“在目标上概率”都会把它反复算进去。

这为什么错。 因为首达时间只允许每条轨迹在**第一次**命中时记账一次。之后它已经完成任务了，不能再重复贡献给 $f(t)$。

正确做法是什么。 正确做法就是：

1. 先让当前未命中质量按一步转移走出去；
2. 统计其中有多少流进了目标，这一部分就是当前的首达质量；
3. 把流进目标的这部分从后续传播里拿掉；
4. 剩下的才是新的未吸收质量，继续下一轮递推。

所以“拿掉 target 概率”不是一个额外技巧，而是**把普通传播改造成 first-passage 传播的本质步骤**。

为什么这仍然是精确的。 因为你并没有扔掉任何有效信息。你只是把“已经完成首达的质量”从“尚未首达的传播池”里移出，并把它记到正确的时间桶里。对于 first-passage 问题，这恰恰是**唯一正确的记账方式**。

为什么 Exact Recursion 不是 Monte Carlo

MC 的做法是抽样很多条随机轨迹，再用频率近似概率。样本数有限时，曲线会有统计噪声，尤其尾部更明显。Exact Recursion 完全不同：它不是抽样，而是直接对概率质量做确定性推进。你可以把它看成“把指数多条轨迹按当前落点无损压缩并求和”。只要状态空间是有限的、转移概率是已知的，那么在给定时间窗内它给出的 $f(t)$ 就是精确的，误差只来自浮点和你主动设定的时间截断。

4 一个极小 toy example

为什么一定要先看小例子。 上面的公式已经是完整正确的，但如果第一次接触 first-passage，最容易卡住的地方不是符号，而是“概率质量到底在怎么流”。所以我们先看一个非常小的图模型，只保留三个状态：

$$A \longleftrightarrow B \longrightarrow T,$$

其中 T 是目标吸收态，A, B 是瞬态态。

具体转移规则。 设从 A 出发：

$$A \to A : 0.2, \qquad A \to B : 0.8, \qquad A \to T : 0.$$

从 B 出发：

$$B \to A : 0.3, \qquad B \to B : 0.1, \qquad B \to T : 0.6.$$

若初始位置在 A，那么

$$u_0 = (1, 0).$$

瞬态子矩阵和吸收流量向量分别是

$$Q = \begin{pmatrix} 0.2 & 0.8 \\ 0.3 & 0.1 \end{pmatrix}, \qquad r = \begin{pmatrix} 0 \\ 0.6 \end{pmatrix}.$$

第 1 步。 由 $u_1 = u_0 Q$ 得

$$u_1 = (1, 0) \begin{pmatrix} 0.2 & 0.8 \\ 0.3 & 0.1 \end{pmatrix} = (0.2, 0.8).$$

这表示：走了 1 步以后，仍然未命中的质量里，有 0.2 在 A，有 0.8 在 B。

而首达概率

$$f(1) = u_0 r = (1, 0) \begin{pmatrix} 0 \\ 0.6 \end{pmatrix} = 0.$$

因为从 A 不能一步直接到目标。

第 2 步。 再递推一次：

$$u_2 = u_1 Q = (0.2, 0.8) \begin{pmatrix} 0.2 & 0.8 \\ 0.3 & 0.1 \end{pmatrix} = (0.28, 0.24).$$

这意味着到第 2 步以后，仍未命中的质量剩下 0.52。

这一轮的首达概率是

$$f(2) = u_1 r = (0.2, 0.8) \begin{pmatrix} 0 \\ 0.6 \end{pmatrix} = 0.48.$$

也就是说，有 48% 的路径是在第 2 步第一次命中目标。

survival 怎么看。 第 1 步 survival 是

$$S(1) = 0.2 + 0.8 = 1.$$

第 2 步 survival 是

$$S(2) = 0.28 + 0.24 = 0.52.$$

和首达质量的关系也很直观：

$$S(2) = S(1) - f(2) = 1 - 0.48 = 0.52.$$

这正是“流进目标的质量被拿掉以后，剩余的未命中质量继续传播”。

这个小例子告诉了我们什么。 它告诉我们：Exact Recursion 并不是在“追踪某一条轨迹”，而是在追踪**所有未命中轨迹的总概率质量怎样在瞬态态空间里流动**。只要你理解了这个 toy example，再回头看二维格点，其实只是状态数更多、边更多、几何更复杂，但数学结构没有变。

5 映射到仓库代码：它到底是怎么实现的

第一步：先把几何转成一步转移。 仓库不会直接在 LaTeX 里求概率，而是先把几何、bias、sticky、barrier、membrane 等规则全部转成一步转移表。对二维主线，核心文件是

`packages/vkcore/src/vkcore/grid2d/model_core_reflecting.py`

它做的第一件事，是对每个格点列出所有允许的一步跳转，并附上对应概率。这样一来，“物理几何”就被翻译成了“离散状态图 + 边权重”。

第二步：把目标边拆出去。 同一个文件里，`build_exact_arrays` 会把所有非目标格点收集成瞬态态列表，然后对每个瞬态态 i 做两类拆分：

- 如果一步会走到目标，就把这条边的概率累积到 r_i；
- 如果一步还是走到非目标格点，就把这条边记到 `src\_idx/dst\_idx/probs` 里。

这一步正是在把整个一步转移矩阵分解成“瞬态到瞬态的 Q”和“瞬态到目标的 r”。

第三步：做稀疏推进。 在 one-target 的通用实现里，主循环大致等价于：

当前未命中分布 p 先按照所有瞬态边推进到 p_{next}；
读出其中流入目标的质量作为 `hit\_total`；
把目标位置清零；
剩余部分作为下一轮继续递推的未吸收质量。

仓库中的通用 one-target 实现可以在

`packages/vkcore/src/vkcore/grid2d/rect_bimodality/cli.py`

里的 `run_exact_one_target_rect` 找到。

为什么这等价于稀疏矩阵乘法。 如果真的把 Q 显式存成一个大矩阵，每一轮只是在做一次向量乘矩阵：

$$u_{t+1} = u_t Q.$$

代码里不显式存整块 Q，而是只存非零边：

`src_idx`, `dst_idx`, `probs`.

然后用“把每条边贡献加到目标节点”的方式完成同样的求和。这其实就是稀疏矩阵乘法的散点累加写法。它的好处是：状态多的时候不必浪费空间存大量零元。

为什么说这仍然是 exact。 这里的“稀疏”只是存储形式稀疏，不是数学近似稀疏。只要所有非零边都完整地被枚举出来，那么这和显式乘上完整矩阵 Q 的结果完全一样。换句话说，**稀疏实现改变的是速度和内存，不改变数学答案。**

6 主线真实例子：shared symmetric one-target baseline

为什么选这个例子。 因为它是你当前仓库 one-target 主语言的中心。它不是某个极端 showcase，而是标准配置。主线报告里已经明确说了：canonical one-target 语言回到了 shared symmetric baseline，第一峰是 corridor fast branch，第二峰的主体是 left-open detour/rollback branch，膜穿越只是慢支路的次级修饰。

这个例子里状态空间长什么样。 主图见图 1。如果用最朴素的话来说，这个例子有五层结构：

1. 一个从左向右的 corridor，是最快的主通道；
2. corridor 上下都有 outside 区域，可以形成更长的绕行；
3. corridor 与 outside 的交界上有半透膜，它会改变进出 outside 的难易；
4. 左端附近有一个起始盆地 A，轨迹是否离开又返回 A 会决定 rollback 类别；
5. 右端有 target，是唯一吸收点。

Exact Recursion 在这里具体输出什么。 最基本的输出是首达时间 pmf

$$f(t) = \mathbb{P}(T = t),$$

也就是命中 target 的时间分布。主线里你看到的 `peak1`、`valley`、`peak2`，都是在这条总曲线上定义的窗口，而不是额外另造出来的对象。

为什么第一峰会早。 如果一条轨迹大部分时间都待在 corridor 里，几乎不做 outside detour，也不发生明显 rollback，那么它会以较短时间到达 target。这批轨迹集中堆在较早的时间上，于是形成 `peak1`。这就是主线里说的 corridor fast branch。

为什么第二峰会晚。 如果一条轨迹在 hitting 之前跑到 outside，或者更典型地，它先离开起始盆地 A，后来又回到 A，那它需要花更长时间才命中 target。这些更慢的路径质量堆在较晚时间上，于是形成 `peak2`。在当前主线语言里，第二峰的主体是 left-open detour/rollback branch，而不是“纯穿膜峰”。

膜穿越在这里扮演什么角色。 膜穿越当然重要，因为它会改变进入 outside 和从 outside 回到 corridor 的难易，从而改变慢支路的组成比例。但主线现在的判断很明确：膜穿越是 slow branch 的次级修饰，而不是主导第二峰存在与否的唯一机制。

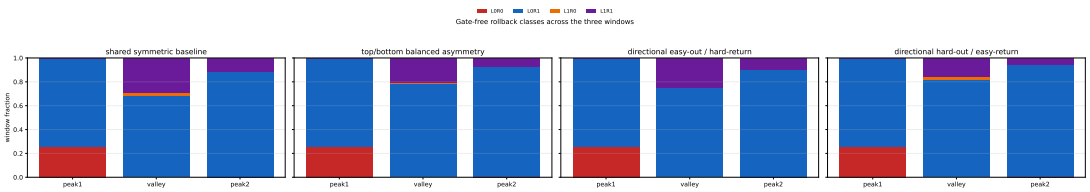

图 2: 主线报告中的 rollback 窗口分解图。它直观展示了：`peak1` 主要由快支路支撑，而 `valley` 与 `peak2` 的质量更多地流向了包含 detour / rollback 的类别。

为什么 Exact Recursion 非常适合这个例子。 因为你不只想知道总曲线是不是双峰，还想知道：

- 每个窗口的质量由哪些路径类别组成；
- 什么时候第一次离开 corridor；
- 什么时候第一次穿膜；
- 在 valley 和 `peak2` 里，outside 停留时间预算有多大差别。

这些都可以建立在同一个“精确推进未吸收质量”的框架上，只是多记录一些事件标签、多做一些窗口条件化统计而已。主线后续的 `grid2d_one_target_window_measures`、`grid2d_one_target_exit_timing` 和 `grid2d_one_target_valley_peak_budget` 都是在这条基础线上往外长出来的。

7 AW 方法是什么，为什么很多人会把它和 Exact Recursion 放在一起比较

先说最直观的区别。 Exact Recursion 是在时间域里一轮一轮推进；AW 方法则是先到变换域，把整条时间分布的生成函数写出来，再把系数反演回时间域。所以两者都能得到 $f(t)$，但走的路径不同：

- Exact Recursion：先有每一步的概率流，再拼出整条曲线；
- AW：先有整条曲线的生成函数，再用数值反演取出每个时间系数。

什么是生成函数。 对首达 pmf

$$f(1), f(2), f(3), \ldots,$$

它的生成函数定义为

$$\widetilde{f}(z) = \sum_{t \geq 1} f(t) z^t. \tag{14}$$

你可以把它想象成“把整个时间序列压缩进一个关于 z 的函数里”。一旦这个函数能被高效计算，就有机会反过来把每个系数 $f(t)$ 再取出来。

propagator 是什么。 propagator 可以理解成“从一个起点到另一个位置，在给定时间或变换域下传播过去的权重”。如果在时间域里写，它长得像

$$P_{ij}(t) = \mathbb{P}_i(X_t = j).$$

如果写成生成函数，就会变成

$$\widetilde{P}_{ij}(z) = \sum_{t \geq 0} \mathbb{P}_i(X_t = j) z^t.$$

对于很多具有高对称性的模型，例如某些 ring 或 defect 很少的结构，$\widetilde{P}_{ij}(z)$ 可以写得很漂亮，这正是 Giuggioli/Sarvaharman 这一类 propagator-renewal 路线常常能发挥威力的原因。

single-target renewal 关系。 如果目标只有一个点 m，那么最常见的关系是

$$\widetilde{f}_{i\to m}(z) = \frac{\widetilde{P}_{im}(z)}{\widetilde{P}_{mm}(z)}. \tag{15}$$

这条式子的直觉是：从 i 传播到 m 的所有路径，可以按“第一次命中 m 是在什么时候”切开。第一次命中之后，后面在 m 附近的传播由 $\widetilde{P}_{mm}(z)$ 接上。把这个 renewal 关系解出来，就得到单目标首达的生成函数。

AW/Cauchy-FFT 反演。 一旦你有了 $\widetilde{f}(z)$，接下来要做的是“取出 z^t 的系数”。最一般的思路是 Cauchy 系数公式：

$$f(t) = [z^t]\widetilde{f}(z) = \frac{1}{2\pi i}\oint_{|z|=r} \frac{\widetilde{f}(z)}{z^{t+1}}\,dz. \tag{16}$$

AW 或离散 Cauchy-FFT 的做法，就是把这条圆周积分离散化成一圈采样点：

$$f(t) \approx \frac{r^{-t}}{m}\sum_{k=0}^{m-1} \widetilde{f}\Big(re^{2\pi ik/m}\Big)\, e^{-2\pi ikt/m}. \tag{17}$$

它看起来像 Fourier 系列，是因为本质上就是在圆周上做离散频率采样，再把系数倒回来。

这些参数是什么意思。

- r 是取样圆的半径。太接近 1 时，系数提取更稳定于长尾，但数值条件可能变差；太小又会放大截断误差。
- m 是圆周上的采样点数。采样点更多，通常反演更精细，但每次都要评估 $\widetilde{f}(z_k)$，总成本会上去。
- 如果每个 z_k 上的 $\widetilde{f}(z_k)$ 本身要通过解一个线性系统才能得到，那真正昂贵的往往不是 FFT，而是每个采样点上的前置求解。

为什么 AW 经常拿来和 Exact 对比。 因为它们解决的是同一个问题：都想得到首达时间分布 $f(t)$。所以非常自然地会问：

> 同一个模型下，是直接在时间域一轮一轮推更划算，还是先在变换域把 $\widetilde{f}(z)$ 写出来再反演更划算？

你的仓库之所以要单独做 `luca_vs_recursion_unified_benchmark`，就是在系统地回答这个问题。

8 Luca/GF 家族到底是什么

不要把它想成一个单独公式。 在你的仓库语境里，Luca/GF 家族更像是一整条“方法链”：

1. 先写 defect-free propagator 或闭式 Green function；
2. 再把局部 defect、door、sticky、shortcut 等有限异质性压缩成低维修正；
3. 通过 Woodbury、determinant closure 或 resolvent 恢复 defected propagator；
4. 再做 single-target 或 multi-target renewal，得到 first-passage 的生成函数；
5. 最后用 AW/Cauchy-FFT 把时间系数反演出来。

为什么这条路线会有时非常强。 如果问题有很好的解析对称性，而且 defect support 很小，那么“先有一个漂亮的无缺陷传播子，再做一个低维修正”就会非常划算。因为你不需要在整个巨大状态空间上逐步推进，只需要在少量 defect 自由度上解问题，然后再把答案拼回来。

为什么它有时又会很贵。 一旦 defect 不再稀疏，或者需要对很多 z_k 做同样昂贵的线性求解，这条路线的成本就会迅速抬高。你的 unified benchmark 已经把这一点总结得很清楚：在有解析 PGF 的单目标 ring，或 ultra-sparse defect 的某些 two-target 工作负载里，Luca/GF 可以赢；但在二维主线、encounter、reflecting 中等或高 defect 的案例里，time recursion 往往更实用。

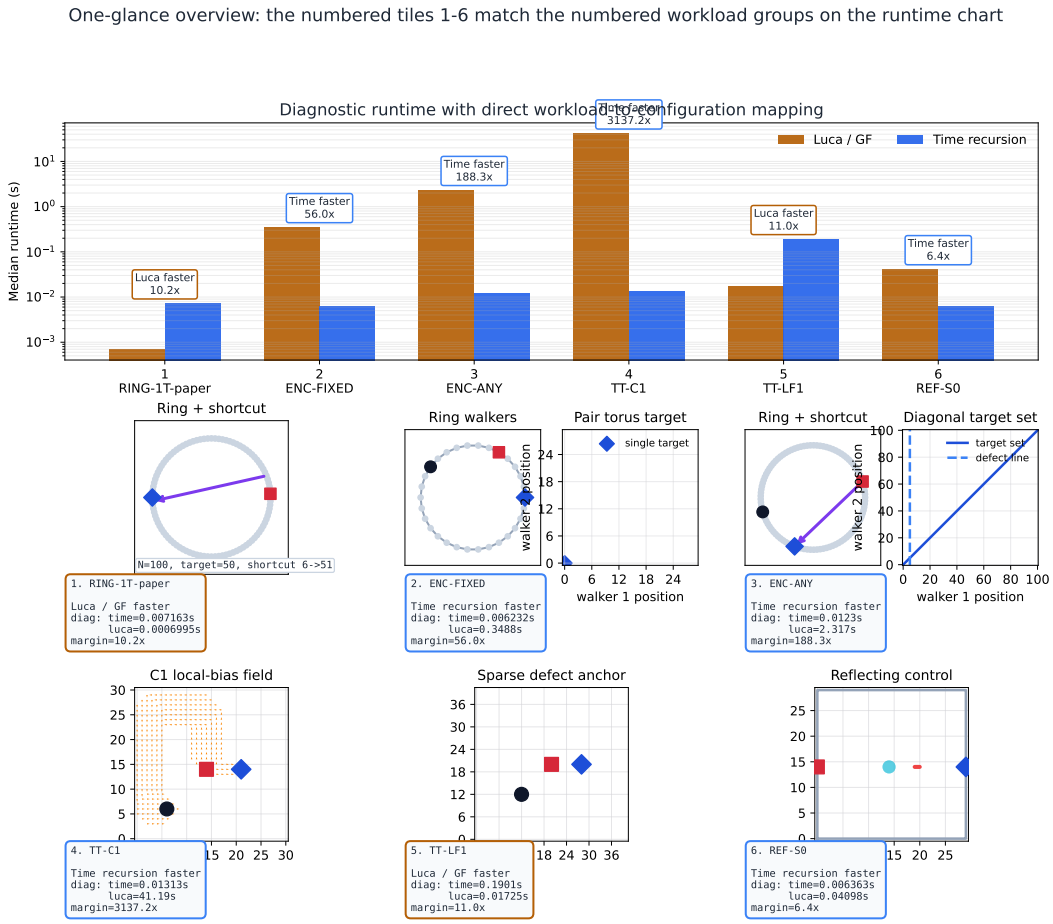

图 3: 统一 benchmark 的配置概览图。它提醒我们：仓库里并不是只研究一种模型，因此不会存在一条在所有工作负载上都最优的方法路线。

为什么 ring 单目标是 Luca/GF 的正锚点。 因为那类模型往往能把生成函数写得很干净。例如 `ring_lazy_flux` 这条线就明确是“Chebyshev 解析生成函数 + AW 反演；flux 交叉验证”的结构。这里 AW/Luca 并不是陪跑，而是很自然的主力方法之一。

9 Exact Recursion、AW、Luca/GF、MC 到底怎么比较

先给一张总表。

方法	直接求的对象	是否精确	主要误差来源	在本仓库里的典型角色
Exact Recursion	未吸收分布 u_t 与首达 pmf $f(t)$	是，在给定时间窗内精确	浮点误差、时间截断	当前二维主线的默认生产器；也是 encounter 与多数 reflecting / two-target 工作负载的实务首选
AW 反演	首达生成函数 $\widetilde{f}(z)$ 的时间系数	反演本身是数值近似，但可很高精度	采样点数 m、半径 r、条件数、截断	在有良好生成函数表达时很强；常用于和 exact 做交叉验证
Luca/GF 家族	propagator / Green function，经 renewal 得到 $\widetilde{f}(z)$	若前置解析和数值求解都正确，可高精度	defect-reduced 解的代价、每个 z_k 的线性求解、反演误差	单目标 ring、极稀疏 defect 的正锚点；统一 benchmark 的重要对照路线
Monte Carlo	轨迹样本的经验频率	否，是统计估计	样本噪声、尾部方差、binning/smoothing	主要用于解释、可视化和 sanity check，而不是主曲线生产器

为什么你的二维主线主要用 Exact Recursion。 因为它同时满足三件事：

1. 目标是完整首达曲线和窗口结构，而不是只要一个平均值；
2. defect / membrane / local bias 的结构已经复杂到让“每个 z_k 做一轮大求解”的代价变得不划算；
3. 你后续还要做大量 window-conditioned observables 和机制分解，而 time recursion 很自然就能和这些统计挂起来。

为什么 ring 的某些解析单目标例子更适合 AW/Luca。 因为那一类模型的闭式 propagator 和单目标 renewal 关系都很干净。一旦生成函数已经在手，反演整条曲线会非常自然。这也是为什么 unified benchmark 里 RING-1T-paper 是 Luca/GF 的正锚点。

为什么 MC 在你的工作里不是主生产器。 这并不是说 MC 没用，而是它的角色不同。你的主结果经常需要：

- 精确辨认双峰是否存在；
- 比较 valley 和 second peak 的细微差异；
- 稳定输出 timing ratio、outside budget、post-crossing budget；
- 在大量参数点上重复跑并保持一致口径。

这些任务如果全靠 MC，样本量会很大、尾部会比较吵、窗口边界也不稳定。Exact Recursion 则更适合作为 deterministic baseline，MC 更像辅助解释层。

10 这套方法怎样贯穿你的研究主线

不是只算一条曲线。 对你的研究来说，Exact Recursion 的价值不在于“把首达曲线画出来”，而在于它提供了一块可重复使用的基底。只要你能把事件条件、区域标签或扩展状态编码进状

态空间或后处理，就能在同一框架里继续计算很多机制量。

它怎样支撑 one-target companions。

- 在 `grid2d_one_target_window_measures` 里，它支撑了窗口条件化的 occupancy share 和 ever-visit probability；
- 在 `grid2d_one_target_exit_timing` 里，它支撑了 τ_{out} 和 τ_{mem} 的窗口条件 CDF、timing ratio 和 early/late/no-exit 分解；
- 在 `grid2d_one_target_valley_peak_budget` 里，它继续支撑 valley 与 `peak2` 的 outside-time share、outside budget 和 post-crossing budget 对比。

这些看起来是不同问题，但底层共同点都是：先得到足够精确的首达结构，再在此基础上做条件化和记账。

它怎样和 benchmark 连起来。 `luca_vs_recursion_unified_benchmark` 做的不是推翻这条主线，而是把它放进更大的方法比较语境里：什么时候应继续用 time recursion，什么时候应该切换到 Luca/GF。也就是说，benchmark 不是反对 Exact Recursion，而是在帮你说明它的优势边界和局限边界。

11 常见误解

误解 1：你主要是不是在做 Monte Carlo？ 不是。MC 在你的仓库里主要承担辅助角色：做代表轨迹、画 overlay、做少量 sanity check。真正的大量主曲线和机制量，尤其是二维主线，并不是靠 MC 样本频率去生产的。

误解 2：你是不是主要用 Luca/GF？ 也不是。Luca/GF 是仓库里非常重要的一条方法线，而且在某些单目标 ring 或稀疏 defect 情况下很强，但它不是当前二维主线的默认生产器。你现在的主线，更准确地说是“以 Exact Recursion 为默认求解器，再用 AW/Luca 做局部正锚点、交叉验证和方法比较”。

误解 3：你的研究是不是纯 lazy？ 不是全都 lazy。仓库整体上既有 lazy 也有 non-lazy 线；只是当前二维主线常用 baseline 的确是 lazy 参数设置，例如很多例子固定 $q = 0.8$。方法本身并不依赖“必须 lazy”，它只依赖你把一步转移概率写清楚。

误解 4：‘exact’到底精确在哪里？ 它的“精确”不是说“一定有闭式公式”，而是说：在给定有限状态空间和给定时间窗内，它直接精确推进全部概率质量，没有 Monte Carlo 采样误差。实际数值上的误差来源只剩下浮点和你主动设定的时间截断。

12 阅读指南与最小复现入口

如果你只想最快读懂主线。 建议顺序是：

1. 先读本文第 2–5 节，真正理解 Exact Recursion 在算什么；
2. 再对照 `grid2d_one_two_target_gating` 的 one-target shared symmetric baseline；

3. 然后读 `grid2d_one_target_exit_timing` 和 `grid2d_one_target_valley_peak_budget`，看同一套递推怎样延展出 timing 和 budget 量；
4. 最后再回头看 `luca_vs_recursion_unified_benchmark`，理解为什么同一个仓库里需要两条方法线。

与仓库代码最相关的入口。

- 二维主线状态与转移构造：`packages/vkcore/src/vkcore/grid2d/model_core_reflecting.py`
- one-target 精确递推骨架：`packages/vkcore/src/vkcore/grid2d/rect_bimodality/cli.py`
- 当前主线报告：`research/reports/grid2d_one_two_target_gating/`
- 统一方法比较：`research/reports/luca_vs_recursion_unified_benchmark/`

最小复现命令。 对本文自身：

```
python3scripts/reportctl.pybuild--reportexact_recursion_method_guide--langcn
```

对主线 one-target 机制稿：

```
python3scripts/reportctl.pybuild--reportgrid2d_one_two_target_gating--langcn
```

对统一 benchmark：

```
python3scripts/reportctl.pybuild--reportluca_vs_recursion_unified_benchmark--langcn
```

参考文献

[1] J. Abate and W. Whitt, *The Fourier-series method for inverting transforms of probability distributions*, Queueing Systems **10**, 5–88 (1992).

[2] S. Sarvaharman and L. Giuggioli, *First-passage-time statistics of a random walker under biased quenched disorder*, Phys. Rev. E **102**, 062124 (2020).

[3] L. Giuggioli and S. Sarvaharman, *Spatio-temporal dynamics of random transmission events: from information sharing to epidemic spread*, J. Phys. A: Math. Theor. **55**, 375005 (2022).

[4] L. Giuggioli, S. Sarvaharman, D. Das, D. Marris, and T. Kay, *Multi-target search in bounded and heterogeneous environments: a lattice random walk perspective*, arXiv:2311.00464v2 [cond-mat.stat-mech] (2024).